دانشگاه تهران
دانشکدگان فنی

برنامه‌سازی پیشرفته

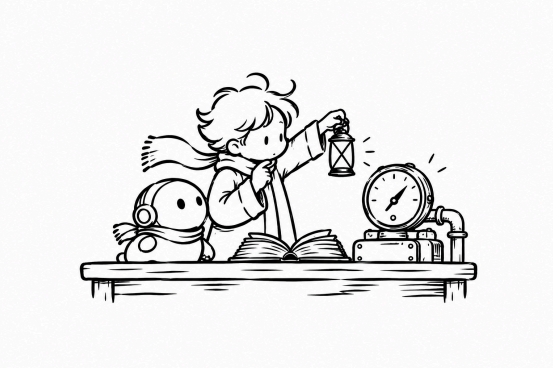
پیش‌نویس جزوهٔ دانشجو — گزینهٔ الف

دفتر فانوس‌بان: نشانِ نادرست

چهار ایستگاه برای دیدن مسئله، یافتن شاهد و آزمودن یکِ تغییر

آغاز دفتر

فانوس‌بانِ ایستگاه «نصف‌النهار» هر شب با یک سؤال شروع می‌کند: «قرار بود چه ببینم و حالا چه می‌بینم؟» چراغ، بندرَ و نقشهٔ مسیر به هم مربوط‌اند؛ اما هر نشانه را جداگانه بررسی می‌کند. هر قطعهٔ کد را در اجرای جداگانه و در بخش آمادهٔ برنامه بررسی کنید. فقط دستورهای مربوط به مقدار، تصمیم، تکرار و رسم را دنبال کنید.



۱. عددی که روی صفحه ننشست

برای باتریِ 7 واحدی از ظرفیت 20، انتظار، نمایش 35.0 درصد است. اما برنامه هنوز اجرا نشده است؛ کامپایلر کنار این خط می‌ایستد. این **خطای زمانِ کامپایل** است، پس پیش از حدس‌زدن دربارهٔ خروجی، متن پیام و خط نشان‌داده‌شده را بخوانید.

```
int remaining = 7;
int capacity = 20;
int batteryPercent = 35.0; // intentional compile error
```

```
error: incompatible types: possible lossy conversion from double to int
```

فانوس‌بان می‌پرسد: «نوع سمت راست چیست؟ نوع سمت چپ چه چیزی می‌پذیرد؟» برای بازتولید، همین سه خط کافی است. هنگام جست‌وجو، عبارت `Java possible lossy conversion from double to int` را با متن واقعی خطا بنویسید، نه نام پوشه یا اطلاعات شخصی پروژه را.

سرنخ‌های کنار دستگاه

این هشت مقدار، نمونه‌های معتبرِ نوع‌های پایه‌اند. `L` پسوند `long`، `f` پسوند `float` و تک‌نقل‌قول نشانهٔ یک `char` است.

```
byte signal = 12;        short steps = 240;
int remaining = 7;       long distance = 3000000000L;
float ratio = 0.5f;      double percent = 35.0;
char marker = 'M';       boolean lampOn = true;
```

اعلان، نام و نوع را معرفی می‌کند؛ مقداردهی اولیه نخستین مقدار را می‌دهد. `final` مقدار را پس از مقداردهی ثابت نگه می‌دارد. نامی که داخل آکولاد داخلی اعلان شود بیرون آن دامنه وجود ندارد. علامت = مقدار سمت راست را به متغیر سمت چپ می‌سپارد.

```
int charge;               // declaration
charge = 7;               // first value
final int capacity = 20;
{ int used = 2; charge -= used; } // used ends with this block
charge += 3;              // 8
charge *= 2;              // 16
charge++;                 // 17
charge--;                 // 16
```

پنج عمل +، -، *، / و % به‌ترتیب جمع، تفریق، ضرب، تقسیم و باقی‌مانده‌اند. این ردگیریِ کوتاه را با مقدار مورد انتظار مقایسه کنید:

```java
int sum = 7 + 2;              // 9
int difference = 7 - 2;       // 5
int product = 7 * 2;          // 14
int quotient = 7 / 2;         // 3
int remainder = 7 % 2;        // 1
double fraction = (double) 7 / 2; // 3.5
```

تقسیم دو عدد صحیح، بخش کسری را حذف می‌کند؛ برای نتیجهٔ کسری، تبدیل باید **پیش از** تقسیم رخ دهد. تقدم، عبارت `4 - 6 + 3 * 4 + 2` را به صورت `(4 * 3) + 2` گروه‌بندی می‌کند؛ عملگرهای هم‌تقدم در 20 از چپ گروه‌بندی می‌شوند و حاصل 10 است. گروه‌بندی را با **ترتیب ارزیابی عملوندها** یکی ندانید: Java عملوندها را از چپ به راست ارزیابی می‌کند؛ در `n++ + n++` با `n = 1`، دو مقدارِ واردشده به جمع 1 و 2 هستند. حالا در اجرای جدا با `int remaining = 7;` و `int capacity = 20;`، یک چاپ موقت کنار محاسبه بگذارید: `System.out.println(remaining / capacity * 100);`. چه چاپ می‌شود؟ آیا همین، توضیحِ خوانده‌شدنِ 0 درصد روی صفحه است؟ نخست فرضیه را بنویسید، سپس فقط یک بخش عبارت را عوض کنید و همان ورودی را دوباره اجرا کنید.

۲. چراغی که باید سه بار چشمک بزند

در دفتر نوشته‌اند: «سه قاب، با شماره‌های 0، 1 و 2». در `for` آغاز، شرط و افزایش کنار هم‌اند. برای `n = 1`، بدنه فقط یک بار اجرا می‌شود.

```java
int n = 3;
for (int frame = 0; frame < n; frame++) {
    System.out.println(frame);
}
```

همان ترتیب را می‌توان با `while` دید: آغاز یک‌بار؛ سپس شرط، بدنه و افزایش. اگر چشمک بیشتر از انتظار بود، مقدار `frame` را نزدیک انتهای بدنه چاپ کنید و نخستین دورِ اضافه را پیدا کنید.

```java
int n = 3;
int frame = 0;
while (frame < n) {
    System.out.println(frame);
    frame++;
}
```

حالا خواندنِ پیام‌های چراغ تعداد ثابتی ندارد. محیط تمرین، ورودی را با `Scanner` آماده کرده است. `-1` نشانهٔ پایان است؛ اگر نخستین ورودی `-1` باشد، بدنه هیچ‌بار اجرا نمی‌شود. برای `7 4 -1`، جمع باید 11 باشد و نشانه به جمع نمی‌رود.

```java
// supplied input
java.util.Scanner input = new java.util.Scanner(System.in);
int total = 0;
int reading = input.nextInt();
while (reading != -1) {
    total += reading;
    reading = input.nextInt();
}
System.out.println(total);
```

در پوستهٔ آمادهٔ گرافیکی ACM، همان ایدهٔ «هر قاب یک تغییر» دیده می‌شود. `GOval` شکل قایق را می‌سازد؛ `add` آن را روی بوم می‌گذارد. هر دور، `move(2, 0)` قایق را دو واحد به راست می‌برد و `pause(40)` مکث می‌کند. اگر جای قایق نادرست شد، مختصات آغاز، تعداد قاب‌ها و تغییر هر قاب را جدا بررسی کنید.

```java
// inside the supplied ACM run section
GOval boat = new GOval(30, 190, 24, 12);
add(boat);
for (int frame = 0; frame < 3; frame++) {
    boat.move(2, 0);
    pause(40);
}
```

۳. پیام بندر و خطی که برنامه را متوقف کرد

فانوس‌بان می‌خواهد میانگین بارِ هر قایق را بسنجد. وقتی هیچ قایقی ثبت نشده، انتظار دارد تصمیمِ بندر بدون تقسیم انجام شود. ولی با `boats = 0` برنامه متوقف می‌شود. این بار کد کامپایل شده و **خطای زمانِ اجرا** رخ داده است.

```
int total = 18;
int boats = 0;
int average = total / boats; // intentional error when boats = 0
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at HarborDecision.run(HarborDecision.java:18)
    at acm.program.Program.start(Program.java:...)
```

کل ردِ پشتهٔ بالا نمونه‌ای ساختگی برای تمرینِ خواندن است؛ نام فایل، قاب‌ها و شمارهٔ خط از اجرای واقعی گرفته نشده‌اند. نام استثنا می‌گوید چه رخ داده؛ نخستین قابِ مربوط به کد خودتان، `HarborDecision.java:18`، جای بررسی است. قاب‌های پایین‌تر زمینهٔ فراخوانی‌اند. در اجرای خودتان، شماره و متن همان اجرای واقعی را بخوانید. یک چاپ موقتِ `System.out.println("boats=" + boats);` درست پیش از تقسیم، شاهدِ مقدار است. فرضیهٔ خود را با ورودی‌های `boats = 0` و `boats = 3` بسنجید.

دروازهٔ تصمیم

مقایسه‌ها مقدار `boolean` می‌دهند: `==` برابری، `!=` نابرابری، و `<`، `>`، `<=`، `>=` ترتیب را می‌سنجند. `=` انتساب است. این وضعیت‌های کوچک را پیش از اجرای شرط بخوانید:

```
int boats = 3;
boolean empty = boats == 0;     // false
boolean changed = boats != 3;   // false
boolean fewer = boats < 4;      // true
boolean many = boats > 2;       // true
boolean room = boats <= 4;      // true
boolean atLeast = boats >= 3;   // true
```

`&&` به دو شرطِ درست نیاز دارد؛ `||` با یک شرطِ درست هم درست می‌شود؛ `!` نتیجه را وارونه می‌کند. در ارزیابی کوتاه‌مدار، اگر سمت چپ `&&` نادرست یا سمت چپ `||` درست باشد، سمت راست اجرا نمی‌شود. جای آزمون `boats` را با جای تقسیم مقایسه کنید:

```
int total = 18;
int boats = 0;
boolean overLimit = boats != 0 && total / boats > 5;
boolean emptyOrOver = boats == 0 || total / boats > 5;
boolean notEmpty = !(boats == 0);
```

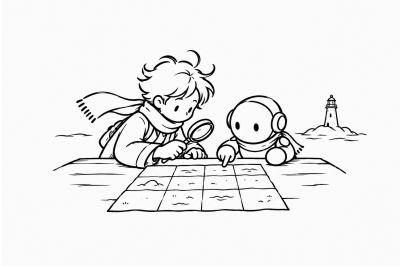
با `boats = 0`، دو عبارت اول بدون تقسیم به‌ترتیب `false` و `true` می‌شوند. با `boats = 3`، سمت راست هر دو بررسی می‌شود. این مثال‌ها جدا از قطعهٔ خطادار بالای صفحه‌اند؛ از روی آن‌ها دربارهٔ علت توقف فرضیه بسازید.

در تصمیمِ نهایی، `if`، `else if` و `else` فقط یک مسیر را انتخاب می‌کنند. `?:` دو مقدار را با یک شرط انتخاب می‌کند. `switch` مقدارِ چندحالته را با `case` می‌سنجد؛ `break` از ادامه به شاخهٔ بعدی جلوگیری می‌کند و `default` حالت‌های دیگر را می‌گیرد.

```
int wind = 18;
int boats = 0;
if (wind >= 30) { System.out.println("closed"); }
else if (wind >= 15) { System.out.println("caution"); }
else { System.out.println("open"); }
String status = boats == 0 ? "empty" : "occupied";
switch (wind / 15) {
    case 0: System.out.println("open"); break;
    case 1: System.out.println("caution"); break;
    default: System.out.println("closed"); break;
}
```

۴. خانهٔ اضافه روی نقشه

فانوس‌بان مسیرِ 4 در 4 می‌خواهد. خانهٔ بالا و چپ با `row = 0` و `col = 0` باید X باشد؛ پس سطر اول `X . X .` و سطر بعد `. X . X` است. ولی چاپ واقعی در هر سطر یک خانهٔ اضافه دارد. این **خطای منطقی** است: کد اجرا می‌شود، اما نتیجه با انتظار نمی‌خواند.



```
int rows = 4, cols = 4;
for (int row = 0; row < rows; row++) {
    for (int col = 0; col <= cols; col++) { // intentional bound error
        System.out.print((row + col) % 2 == 0 ? "X " : ". ");
    }
    System.out.println();
}
```

چهار سطرِ مورد انتظارِ تخته این‌ها هستند:

```
X . X .
. X . X
X . X .
. X . X
```

در اجرای قطعهٔ خطادار، سطر نخست `X . X . X` می‌شود؛ هر سطر یک خانهٔ اضافه دارد.
حلقهٔ بیرونی سطر و درونی ستون را می‌سازد؛ `2 %` زوج‌بودنِ جمع آن‌ها را می‌سنجد. در اجرای جدا، `System.out.println("row=" + row + " col=" + col);` را پیش از چاپ خانه بگذارید و نخستین ستونِ اضافه را بیابید. چه شرطی دقیقاً ستون‌های 0 تا 3 را می‌سازد؟
در بخش آمادهٔ `run` از بوم ACM، `GRect` خانه، `GOval` نشان، `GLine` مرز و `GLabel` برچسب می‌سازند.

```
int cell = 18, row = 1, col = 2;
GRect tile = new GRect(col * cell, row * cell, cell, cell);
GOval mark = new GOval(col * cell + 5, row * cell + 5, 8, 8);
GLine border = new GLine(0, 72, 72, 72);
GLabel title = new GLabel("route", 0, 90);
add(tile); add(mark); add(border); add(title);
```

برای اختلاف چاپ و رسم، `col * cell` و `row * cell` را ردگیری کنید.

یادداشت پایانی فانوس‌بان

کارتِ همراهِ عیب‌یابی
۱. **صورت مسئله:** انتظار، مشاهده و کوچک‌ترین ورودیِ بازتولید را بنویسید.
۲. **شاهد:** پیام کامپایلر، نخستین قاب مرتبط از کد خودتان در ردِ پشته، یا یک `System.out.println` کنار مقدار مشکوک را بخوانید.
۳. **فرضیه:** بگویید کدام مقدار یا شرط باید فرق کند؛ هر بار یک تغییر را آزمایش کنید.
۴. **جست‌وجو و پرسش:** با Java و متن دقیق خطا جست‌وجو و توضیح‌ها را با منابع معتبر مقایسه کنید. نام‌ها، مسیرها، کلیدها و داده‌های خصوصی را حذف کنید. به AI ورودی، کدِ کوتاه، انتظار، مشاهده، خطا و فرضیه را بدهید؛ توضیح و آزمون کوچک بخواهید. برای مفهوم ناآشنا بپرسید «ساده‌تر و با یک مثال توضیح بده»؛ ادعا را با اجرا بسنجید.
۵. **تأیید:** ورودی اصلی و یک حالتِ نزدیک، مثل `boats = 0` و `boats = 3`، را دوباره اجرا و علت نتیجه را ثبت کنید.